

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: CSA1412

COURSE NAME: Compiler Design

COURSE OUTCOME COVERED

CO2: Construct top-down and bottom up parsers using CFG for parsing conflicts and error recovery for efficient syntax tree generation. (BL3)

Assessment Tool Weightage: 40%

QUESTIONS:5

Duration :60 Mins
On Class
Total Marks:50

Annexure A

DESIGN TASK

Code of Conduct:

*I B h a v y a S (1 9 2 5 2 4 3 7 6) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student: Bhavya S

NAME : S. BHAVYA
REG NO : 192524376

COURSE CODE : CSA1412
COURSE NAME : COMPILER DESIGN

C02 - AT1

1. Given Grammar:

$S \rightarrow ABD$

$A \rightarrow a | Db | \epsilon$

$B \rightarrow gD | dA | \epsilon$

$D \rightarrow e | f$

i) First Sets:

Non-Terminal	FIRST
S	{a, d, e, f, g}
A	{a, e, f, ϵ }
B	{g, d, ϵ }
D	{e, f}

ii) Follow Sets:

Non-Terminal	FOLLOW
S	{ $\$$ }
A	{g, d, e, f}
B	{e, f}
D	{b, e, f, $\$$ }

iii) Predictive Parsing Table:

T \ NT	a	d	e	f	g	$\$$
S	$S \rightarrow ABD$	$S \rightarrow ABD$	$S \rightarrow ABD$	$S \rightarrow ABD$	$S \rightarrow ABD$	
A	$A \rightarrow a$	$A \rightarrow \epsilon$	$A \rightarrow Db \epsilon$	$A \rightarrow Db \epsilon$	$A \rightarrow \epsilon$	
B		$B \rightarrow dA$	$B \rightarrow \epsilon$	$B \rightarrow \epsilon$	$B \rightarrow gD$	
D			$D \rightarrow e$	$D \rightarrow f$		

iv) LL(1) Verification:

First / First Conflict: No Conflict

First / Follow Conflict: Yes, for $A \rightarrow Db$ & $A \rightarrow \epsilon$ on terminals e & f .

Conclusion: The grammar is not LL(1) because multiple entries occur in the parsing table.

v) Predictive Parsing (Input: $agde\$$):

Stack	i/p string	Action
\$S	agde\$	$S \rightarrow ABD$
\$DBA	agde\$	$A \rightarrow a$
\$DB	gde\$	Match a
\$DDg	gde\$	$B \rightarrow gD$
\$DD	de\$	Match g
\$D	e\$	$D \rightarrow e$
\$	\$	Accept

vi) Design Recommendation:

→ The grammar cannot be directly used for LL(1) parsing because of the First / Follow conflict.

→ The conflict is caused by the ϵ -production of A.

→ The grammar should be modified to remove the conflict before constructing the parser.

Advantages:

- No backtracking
- Fast parsing ($O(n)$)
- Easy table-driven implementation
- Efficient syntax error detection.

2. Given Grammar:

$$E \rightarrow E + E / E - E$$

$$E \rightarrow E * E / E / E$$

$$E \rightarrow (E)$$

$$E \rightarrow id$$

Input String: $id * id / (id + id)$

i) Stack Implementation:

- Initialize stack with \$.
- Shift: Push input symbol onto the stack.
- Reduce: Replace the handle with E using the grammar.
- Continue until the input is accepted.

11) Parsing Trace:

Stack	Input	Action
\$	$id * id / (id + id) \$$	shift id
$\$ id$	$* id / (id + id) \$$	Reduce $id \rightarrow E$
$\$ E$	$* id / (id + id) \$$	shift $*$
$\$ E *$	$id / (id + id) \$$	shift id
$\$ E * id$	$/ (id + id) \$$	Reduce $id + E$
$\$ E * E$	$/ (id + id) \$$	Reduce $E * E \rightarrow E$
$\$ E$	$/ (id + id) \$$	shift $/$
$\$ E /$	$(id + id) \$$	shift $($
$\$ E / ($	$id + id) \$$	shift id
$\$ E / (id$	$+ id) \$$	Reduce $id + E$
$\$ E / (E$	$+ id) \$$	shift $+$
$\$ E / (E +$	$id) \$$	shift id
$\$ E / (E + id$	$) \$$	Reduce $id \rightarrow E$
$\$ E / (E + E$	$) \$$	Reduce $E + E \rightarrow E$
$\$ E / (E$	$) \$$	shift $)$
$\$ E / E$	$\$$	Reduce $(E) \rightarrow E$
$\$ E / E$	$\$$	Reduce $E / E \rightarrow E$
$\$ E$	$\$$	accept

iii) Handles Recognized:

→ $id \rightarrow E$

→ $E * E \rightarrow E$

→ $id \rightarrow E$

→ $E + E \rightarrow E$

→ $(E) \rightarrow E$

→ $E / E \rightarrow E$

iv) Parser Decisions:

→ The parser performs shift until a valid handle is formed.

→ Then it performs Reduce.

→ Operator precedence followed:

* & / have higher precedence than + & -.

* All operators are left associative.

→ Shift - reduce conflicts are resolved using precedence & associativity.

v) Result:

→ Final Stack: $\$E$

→ Input: $\$$

→ Status: Accepted

Conclusion: The input string $id * id / (id + id)$ is successfully parsed by the Shift - Reduce parser.

3. Given Grammar:

$$S \rightarrow (L) | a$$
$$L \rightarrow SL'$$
$$L' \rightarrow , SL' | \epsilon$$

Input String: (a)

i) Recursive Procedures:

S():

if '(' then match('(', L(), match(''))

else if 'a' then match('a').

else error()

L():

S()

L'()

L'():

if ',' then match(',', S(), L'())

else return.

ii) Parsing Method:

→ Uses Top-Down Parsing.

→ No backtracking.

→ Predictive Recursive Descent Parser

iii) Function Call Sequence:

for input (a)

S()

↓

match('(')

↓

L()

↓

S()

↓

match('a')

↓

L'() → ε → match('')

iv) Acceptance:

Input (a)

All symbols are matched correctly.

Input is completely consumed.

Result: Accepted.

v) Error Detection & Recovery:

→ If an unexpected symbol is found, display "Syntax Error".

→ Skip the invalid symbol & continue parsing (simple error recovery).

Conclusion:

→ The recursive descent parser successfully parses (a).

→ The grammar is suitable for predictive recursive descent parsing b/c it has no left recursive & supports top-down parsing without backtracking.

4. Given Input:

Input String: $id - id * id$

i) Unambiguous Grammar:

$$E \rightarrow E + T \mid E - T \mid T$$

$$T \rightarrow T * F \mid F$$

$$T \rightarrow (E) \mid id$$

ii) Operator Precedence:

Operator	Precedence	Associativity
*	Highest	Left
+, -	Lower	Left
()	Special	-

iii) Operator Precedence Table:

	+	-	*	id	(	)	\$
+	>	>	<	<	<	>	>
-	>	>	<	<	<	>	>
*	>	>	>	<	<	>	>
id	>	>	>			>	>
(	<	<	<	<	<	=	
)	>	>	>			>	>
\$	<	<	<	<	<	<	Accept

iv) Stack-Based Parsing:

Initialize stack using \$.
 Shift symbols until a handle is formed.
 Reduce the handle using grammar rules.
 Continue until accept.

v) Parsing trace:

Stack	Input	Action
\$	id-id*id\$	shift id
\$id	-id*id\$	Reduce $id \rightarrow F \rightarrow T \rightarrow E$
\$E	-id*id\$	Shift -
\$E-	id*id\$	Shift id
\$E-id	*id\$	Reduce $id \rightarrow F \rightarrow T$
\$E-T	*id\$	Shift *
\$E-T*	id\$	Shift id
\$E-T*id	\$	Reduce $id \rightarrow F$
\$E-T*F	\$	Reduce $T * F \rightarrow T$
\$E-T	\$	Reduce $E - T \rightarrow E$
\$E	\$	accept

Conclusion:

- The parser follows operator precedence (* before -).
- Left associativity resolves conflicts.
- The input string id-id*id is successfully accepted by the operator precedence parser.

3. German Grammar:

$$S \rightarrow AA$$
$$A \rightarrow aAb$$

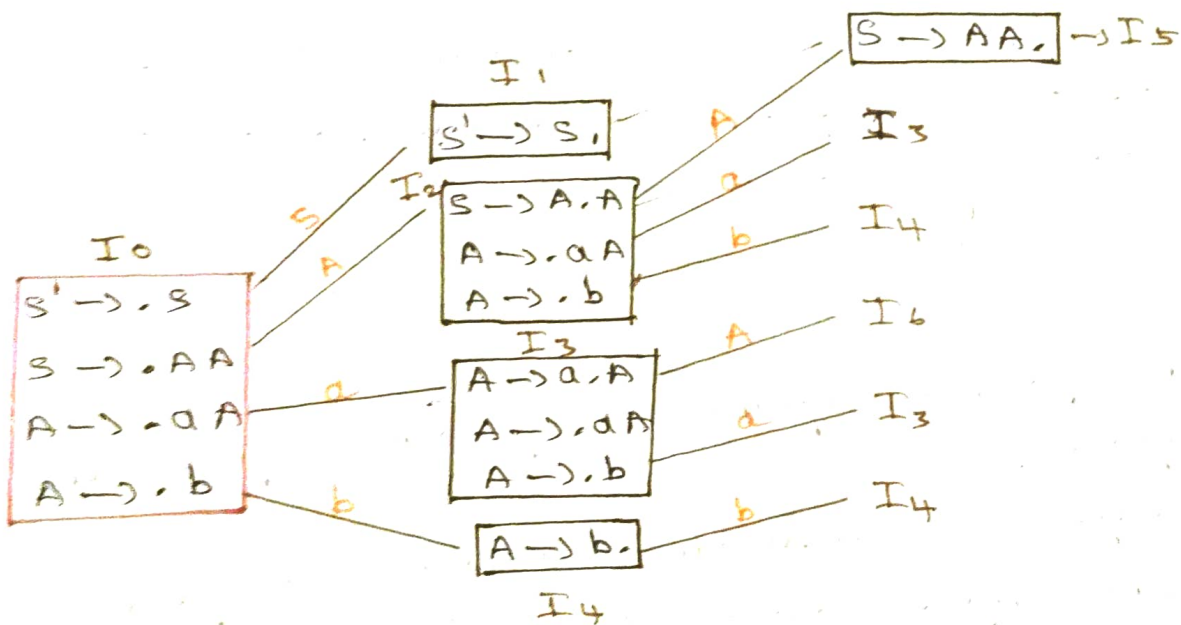
Input String: aabb

i) Augmented Grammar;

5-6

$$S \rightarrow AA$$
$$A \rightarrow aA$$
$$A \rightarrow b$$

ii) Canonical LR(1) Items:



iii) First & Follow Sets:

Non-Terminal	First	Follow
S	{a, b}	{ε, \$}
A	{a, b}	{a, b, \$}

vi) Parsing Table:

	Action			Go to	
	a	b	\$	S	A
I ₀	S ₃	S ₄		1	2
I ₁			accept		
I ₂	S ₃	S ₄			5
I ₃	S ₃	S ₄			6
I ₄	R ₃	R ₃	R ₃		
I ₅	R ₁	R ₁	R ₁		
I ₆	R ₂	R ₂	R ₂		

vii) Parsing Process:

Stack	I/P String	Action
\$0	aabb\$	Shift with S ₃
\$0a3	abb\$	Shift with S ₃
\$0a3a3	bb\$	Shift with S ₄
\$0a3a3b4	b\$	Reduce 3 A → b
\$0a3a3Ab	b\$	Reduce 2 A → aA
\$0A2	b.\$	Shift with S ₄
\$0A2b4	\$	Reduce 3 A → b
\$0A2Ab	\$	Reduce 1 S → AA
\$0S1	\$	accepted

vii) Result:

- the input string aabb is accepted
- the LALR(1) parser successfully parses the input using shift, reduce & accept actions.